L1-计算机发展史

一、计算机类型（Types of Computers）

• 大型机 Mainframe computers (1960s) • 巨型机 Supercomputers (1970s) • 工作站 Workstations (1980s)
• 微型机 Microcomputers (1980s) • 个人电脑 Personal computers (1980s) • 微控制器 Microcontrollers (1980s) (嵌入式或专用计算机，如计算器) • 服务器 Servers (1980s) (通过 network/cloud 可用) • 单片机 Chip computer

随着计算机的发展，性能增长，体积减小(Increased performance & downsizing)。

二、计算机的四个世代（Computer Generations）

第一代：(1944 ~1958)电子管 vacuum tube
第二代：(1959 ~1963)晶体管 transistor
第三代：(1964 ~ 1970)集成电路 IC(integrated circuit)
第四代：(1971 ~ now) 超大规模集成电路 VLSI(Very Large Scale Integration)

三、五种计算机硬件（Hardware）

Input-输入，processing-处理，output-输出，storage-储存，communications-通信

四、计算机软件（Software）

系统软件：与硬件交互（communication），资源管理，便于应用程序之间通信(facilitates communication)
应用软件（Applications software）：便利/辅助用户

五、向后兼容(Backward / Downward Compatibility)新的版本的软/硬件可以使用老版本的软/硬件产生的数据。

六、VHDL(Very-High-Speed Integrated Circuit Hardware Description Language)硬件描述语言-集成电路设计

七、计算机系统层次结构（hierarchy of systems） Hardware (CPU, Graphics, Sound)>Software [Operating Systems (O/S kernel, Win32 API)> User code (e.g. myPrograme.c)]

操作系统的优势：硬件系统的功能由 OS 提供给用户，用户也以此交互硬件。易于编程，保护系统与其他用户，提高系统资源使用的效率与公平性。

八、摩尔定律（Moore's Law） The amount of circuitry (number of transistors) which can be placed on a given chip area approximately doubles every two years. ->A circuit designed 24 months ago can now be shrunk to fit into an area of half the size. 可以放置在给定芯片区域上的电路数量(晶体管数量)大约每两年翻一番

九、软硬件交互发展 1.Windowing interfaces –WIMPs. 视窗-Window、图标-Icon、菜单-Menu、鼠标-Pointing device
2. Internet-Netscape 网景浏览器

L2-计算机架构

〇、以下两种架构的基本组件：硬件(CPU(Central Processing Unit), memory, secondary storage(hard disk, CDs), keyboard, screen)，软件，数据（Data that is being manipulated）

一、机器指令（Machine instructions) CPU 执行机器指令，每个 CPU 都有自己的指令集(instruction set)，通常为 100-200 条指令，没有标准指令集。**分类：** • Input-output: IN, OUT • Data transfer and manipulations: MOV, ADD, MUL, AND, OR • Transfer of program control: JMP, JC, • Machine control: can halt processing, reset the hardware, INT, HLT

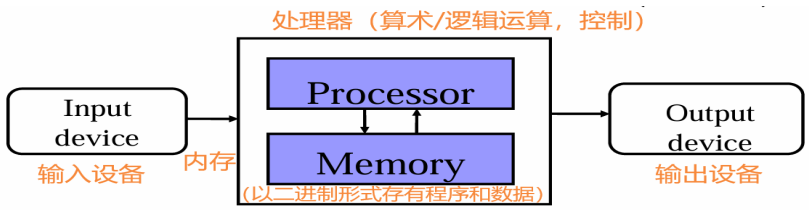
二、HLL 高级编程语言(High Level Programming Language)(: FORTRAN, C, C++, Java, Perl…)

比机器指令更适合编程，但程序仍需翻译为机器代码(硬件只能处理机器指令)。

三、输入-处理-输出模型(Input-Process-Output Model)是冯诺依曼模型的基本方案(essential scheme)

处理过程由一个特殊的、定制的程序控制。The process is to be controlled by a special, custom-made program.

四、冯·诺依曼模型(The von Neumann Model) general-purpose machine controlled by an executable program



(late 1940s)

程序-a list of instructions used to direct a task. 处理器-an active part that executes the program instructions.

数据和指令都存储在内存中不同的区域，将某个内存地址开始的字节全部当作指令，将某个内存地址开始的字节全部当作数据。(使计算机可以区分数据和指令)

冯诺依曼瓶颈：CPU 速度快，内存速度慢，CPU 大部分时间需要等待数据和指令从内存读出/写入

五、哈佛架构(Harvard architecture) Program Memory <->CPU<-> Data Memory（速度快高吞吐-嵌入式/CPU 内）

程序指令存储和数据存储分开的存储器结构，当取指使用程序空间，执指使用数据空间时，可以同时处理。

L3-编译原理(Fundamentals of Compiling)

一、语义鸿沟(Semantic gap)自然语言与电脑指令处理数据方式的巨大差异。

二、三种翻译方式(translation)

1. 编译器（Compilers）：在运行前将高级语言编写的源程序翻译成机器语言(sequence of instructions)。
2. 汇编器（Assemblers）：将汇编语言(助记符如 MOV 等)编写的源程序翻译成二进制代码。
3. 解释器（Interpreters）：在运行时转换高级语言的源程序为机器码，不生成目标文件（on-the-fly）。

三、编译(Compiling)*

-[编辑]>源文件(HLL source file) –[编译]>二进制文件(Binary object file+ Library File) –[link]>可执行文件(Executable file)-[加载 Load]>加载(时报错)　　└编译时报错（根据错误信息在源文件中修改）　　└连接时报错

四、连接(Linking)编译器无法解析对其他模块的引用，连接器将所有二进制部分连接，找不到引用则报错。

五、库文件(library file .dll)仅当使用时加入程序，为程序提供许多功能

六、解释器(interpreters) 接受指令，一次一个，翻译和执行交错进行。

C 源代码→编译器（程序）→汇编语言→汇编器→机器码→linking & loading→可执行文件

Java 源代码→编译器→Java 字节码 byte code→Java 解释器（JVM 虚拟机）

七、代码共享和重用(Code sharing and reuse) 源文件(Edit)->Compile(macros)->link(Obj libraries)->Build(Dyna libs)

Source-level subroutines and macro libraries(源代码级子程序和宏库)

Pre-translated re-locatable binary libraries(预翻译可重新定位的二进制库)

Dynamic libraries and dynamic linking(动态库和动态链接)

L4-数据与信息表示

Data (raw facts, measurements)->Information (organized into useful representation)->Knowledge (reasoned analysis)

一、熵(entropy)(Claude Shannon, 1948)-信息论 information theory 编码论 coding theory

Entropy of an event X is related to p(X) 理论上**越接近 e 的进制效率越高**，三进制的效率比二进制高

高进制转低进制用短除法，低进制转高进制求∑(Ai＊X^i)(其中 i 是低进制位数，Ai 是每 i 位上的数字，X 是高进制)

二、ASCII (American Standard Code for Information Interchange,美国信息互换标准代码)**占用 7bit 表示 128 个字符 (Printing/control characters)** '0'=48; '9'=57; 'A'=65; 'Z'=90; 'a'=97; **EBCDIC-8bit**; **Unicode(支持多种语言，占 16 位)**

L5-基本数据类型

一、Java 基本数据类型;char 1 byte; byte 1 byte; short 2 bytes; int 4 bytes; float 4 bytes; long 8 bytes; double 8 bytes

二、整数表示 补码用来表示和操作负整数 0b00000101=5; 0b11111011=-5；**正数的原/反/补码一模一样**

-1=原:[1000 0001] =反:[1111 1110] = 补:[1111 1111] 反码符号位不变，其余各位取反。补码在反码的基础上加 1 0 用[0000 0000]表示；-128 无原/反码，补码是[1000 0000]。计算机实际只存储**补码**，符号位可参与计算。

三、实数表示 IEEE754 (-1)^s＊(1+M)＊2^E-127 (**单**-3.4E38 ~ 3.4E38;**双**-1.79E+308~1.79E+308)

1 位**符号位(Sign)S**+8/11 位**偏移后的指数位()E**(-127/-1023)+23/52 位**尾数位(fraction)M**

四、变量申明(Declaration of variables) 告诉编译器以正确的内存空间保存变量，要使用什么编码/解码/表示方案

L6-操作系统(Operating System)与网络

OS/360, Unix, MS-DOS & Mac OS, Windows 95, 98, NT, 60s'-90s' 核心都是 **windowing interfaces**

一、操作系统的作用 Management:• To control and operate hardware in an efficient way. 管理系统资源

Provide functionalities allow the user: •efficient access to the facilities of the machine. 方便用户调用系统资源

- fair access to the facilities of the machine. 允许用户公平的使用系统资源
- protected access to the facilities of the machine. 保护系统资源 • Interaction with the user. 与人交互

二、操作系统分层模型(Onion ring model) User apps(Command-Line Interface 命令行(kernel (Hardware

三、现代操作系统 •Multi-tasking 允许程序同时运行-Dos 只能同时运行一个程序 •Multi-user 允许多用户同时操作

系统交互方式 DOS: type command in command line. Unix: shell scripts. Win/Mac OS: click with mouse on icons.

User → command interface→file services→I/O services→computer hardware **操作系统（OS）示意图**

```
↓                    OS      ↑          ↑
Application program-----------┴---------┘
```

四、计算机网络-三次握手 1st 客户端(Client)发送网络包(request message)，服务端(Server)收到了。服务端得出结论：客户端的发送能力、服务端的接收能力正常。**2nd** 服务端发包，客户端收到了。客户端得出结论：服务端的收发能力，客户端的收发能力正常。但此时服务器并不能确认客户端的接收能力是否正常。**3rd** 客户端发包，服务端

收到。服务端得出结论：客户端的收、发能力正常，服务器自己的发、收能力也正常。

L7-总线，指令集，指令周期，接口

〇、主要组件:CPU, Main memory, IO units

一、处理器和寄存器 Processor (arithmetic/logic unit (ALU), control unit-**machine cycle**)

Registers(Program counter(PC)-下条指令地址, Instruction register(IR)-存储当前正在执行的指令)

二、总线(buses)每个硬件单位连接到总线且相互通信，加入/更换新设备时几乎没有中断(disruption);

n 个点点对点链接，需要 n(n-1)/2 路径，总线只需要一条，但只能在同一时间传输一个项目(Bus Bottleneck)

三、机器周期：指令集(Instruction Set) **fetch-execute cycle/machine cycle（获取-执行周期）**

I-cycle (Instruction fetch→Instruction decode)→E-cycle(Execute→Store results)

四、复杂指令集 CISC-complex instruction set 大多数电脑使用 **精简指令集 RISC**-reduced instruction set

RISC 往往更快 芯片空出区域可用来加速常用指令性能，更易优化设计，简化从 HLL 到较小指令集的翻译

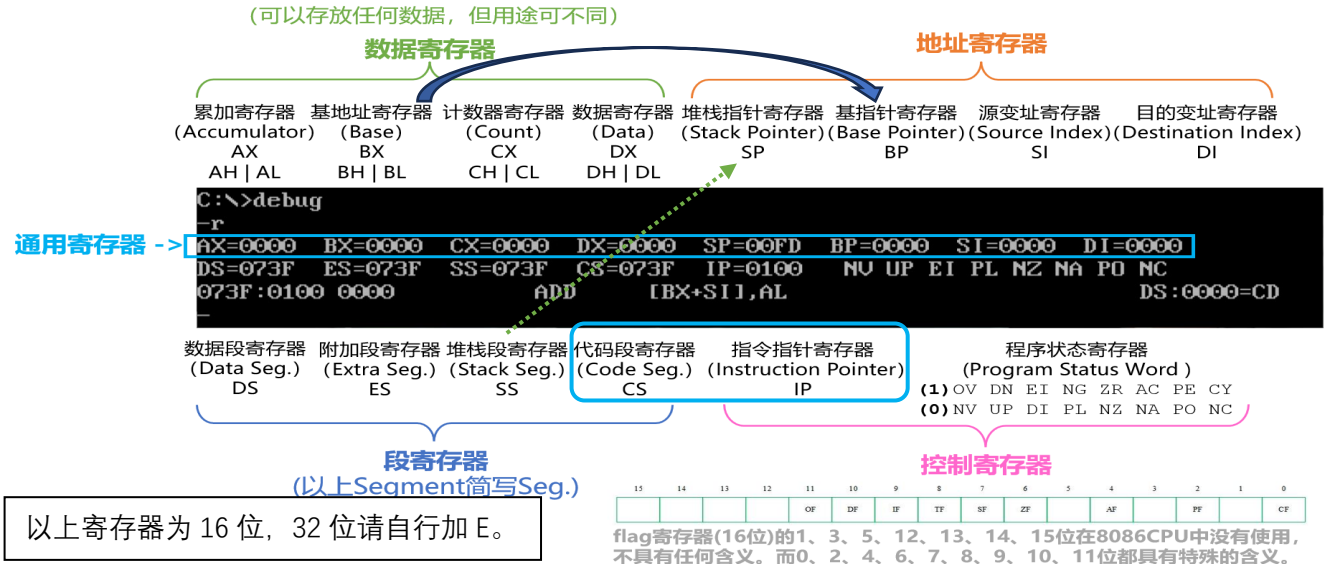
五、通信硬件 Parallel port 并口 IEEE 1284 printers, some scanners **Serial port 串口** RS-232 modems, scanners, mice

L8~9-汇编语言与汇编器

〇、汇编语言程序-[汇编器 Assembler]>二进制机器码（binary Machine code）同时将 labels 与内存地址相关联

一、labels L1: JZ L2 … 表示一块储存单元的开始/loop 循环的开始等

二、Binary machine object code program 也被叫做 **object file**



以上寄存器为 16 位，32 位请自行加 E。

栈： x86 作为小端，栈从高地址向低地址增长，栈是从上往下压的。**栈指针寄存器（SS:SP）指示当前栈顶**，而**栈的增长是由 SP 减小实现**的，**先进后出，add esp,4 清栈**，X:BP 指向栈底。**只有 4 个寄存器能以 [⋯] 的方式使用**：当以 [⋯] 方式访问内存，且使用 BP 且未明确给出段地址时，默认用 SS 寄存器中的值（BX,SI,DI 默认用 DS）。

EAX：累加寄存器，是算术运算的主要寄存器，和数据寄存器都可以分为低 16 位，高/低 8 位单独使用

EBX：基地址寄存器，可保存指向数据结构（比如 Arrays）的基地址。

ECX：计数寄存器，loop，rep（重复前缀指令）都是根据 CX 的值进行判断然后决定是否进行跳转；在 LOOP 指令和串处理指令中用作**隐含计数器，每循环一次递减 1，降为 0 前跳<label>。需要注意，如果中间调用函数，可能污染寄存器内容。此时请使用栈临时储存计数器，在循环结束前释放。`Push ECX; call printf; pop ECX; Loop Lj;`**

CS:IP：指令指针，程序运行过程中 **CS:IP 始终指向下一次要取出的指令地址**。

JMP 指令-控制 IP 转移到 label 所指示的地址，即强制执行 --JMP myLabel

ESI,EDI 是两个具有自动增减量功能的变址寄存器指针

FLAGs：CPU 状态字：溢出，控制 si,di 高向低，允许中断，陷阱模式，符号标志，零标志，半进位，偶数，进位

MOV 对于变量，加不加[]都表示取值；对于寄存器而言，无[]取值，有[]取址【此条通用】。LEA 对于变量，有无[]都表示取变量地址，相当于指针。对于寄存器而言，无[]取址，有[]取值。

EBP:栈帧基指针，指示当前**栈帧**的基地址

L10-指令结构，寻址模式（Addressing modes）

〇、指令结构[Instruction Prefixes 前缀]+Opcode 操作码+[ModR/M&SIB]+[Displacement 偏移]+[Immediate 立即数]

一、寻址模式 •Immediate (operand) mode：立即数，直接嵌入指令中的常数，无需寻址 `mov eax,104`

•Data Register Direct：寄存器直通，速度**最快** `mov eax, ebx`

•Memory Direct：内存直通 mov eax, a 将变量地址存入临时寄存器再从内存中取对应数据
•Address Register Direct：地址寄存器直通 lea eax, message1 常用于初始化指针，指向数组
•Register Indirect：寄存器间接 mov eax, [ebx]将 ebx 地址赋给 eax
•Indexed Register Indirect with displacement：索引寄存器位移 mov eax,[table + esi] 或 mov eax, table[esi]

L11-printf，jcondition

一、读取一个数字并打印出来 %c-char %d 或%i-带符号的十进制数 %s-字符串直到空格/终止符

```
int main(void)
{
    char message [] = "Your number is %i\n"; // 申明message
    char format [] = "%d"; // 申明 被用于 scanf 的 format string 作为第一个参数
    int input; // 申明用户输入的 int 变量
    _asm {
        lea eax, input
        push eax
        lea eax, format // eax 保存 format string 地址
        push eax // 将 format string 地址入栈
        call scanf // 调用 scanf,会使用栈中的两个参数; scanf (%d, &input);用户输入放入后者
        add esp,8 // 清除栈中的两个位置
        push input // 将 input 的值入栈
        lea eax, message // 将字符串 message 的地址保存在 eax
        push eax // eax 的值入栈
        call printf //调用 printf, 从栈中调用两个参数; printf ("Your number is %\n", input);
        add esp,8 // 清除栈中的两个位置
    }
    return 0;
}
```

二、jcondition 条件跳转 先 CMP eax , ebx->改变 Flag ， 相等时 ZF=0

JE/JZ 相等或为 0 跳转 ZF	JLE 1<=2 跳 OF, SF, ZF	JXCZ 如果 CX =0 则跳 none
JNE 不相等跳转 ZF	JL 1<2 跳 OF, SF	JC 如果进位则跳转 CF
JG 1>2 跳转 OF, SF, ZF	JGE 1>=2 跳 OF, SF	JO 如果溢出则跳转 OF

L12-控制程序流-循环

```
if (c > 0) pos = pos + c;
    else neg = neg + c;
```

```
            mov eax, c
            cmp eax, 0
            jg positive
negative:   add neg, eax
            JMP endif
positive:   add pos, eax
endif:      … …
```

----------此 Java-if 等效于----------

```
for (int x = 0;x < 10; x++)
{
    y = y +x;
}
```

```
            mov eax,0
for_loop:   add y, eax
            inc eax
            cmp eax,10
            JL for_loop
```

----------此 Java-for 等效于----------

```
Do {
    fib0 = fib1;
    fib1 = fib2;
    fib2 = fib1 + fib0;
} while (fib2 < 1000)
```

```
while:      mov eax, fib2
            cmp eax, 1000
            JGE end_while
            mov eax, fib1
            mov fib0, eax
            mov eax, fib2
            mov fib1, eax
            add eax, fib0
            mov fib2, eax
            JMP while
end_while:
```

----------此 Java-do-while 等效于------

层次越向下，容量增加，访问时间增加，降低 CPU 访问频率，每 bit 成本下降

L13-子进程，调用，返回

一、CALL 调用原理 记录当前 EIP 值作为返回地址入栈，后赋 EIP 为子程序地址，子程序执行完毕后返回
二、RET(urn)弹出栈中最后一个储存的地址放入 EIP call <label> … ret

L14-传参-值参数-引用参数

在 Java 中，基本类型（如 int、char 等）是**按值传递**的，而引用类型（如对象、数组等）则是按**引用传递**的。
一、栈帧(stack frame)栈帧是在函数调用过程中动态创建的内存区域，存储局部变量、参数、返回地址、上个栈的 EBP 以支持函数执行和返回。在函数调用时生成，返回时销毁。嵌套函数调用会使栈上形成多个栈帧的**链式结构**。

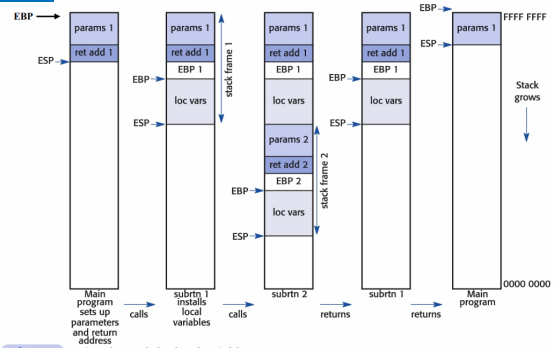


Fig. 8.9 Using thestack for local variables.

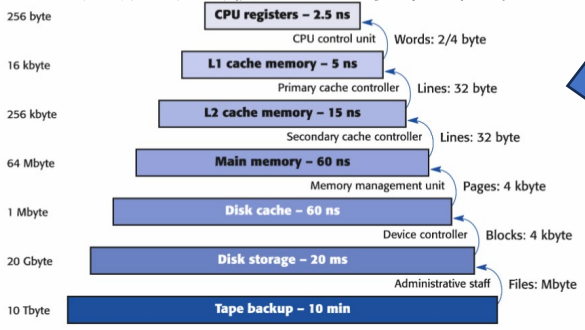


Fig. 12.2 Memory performance and storage hierarchy.

L15-递归

```
static long factorial (int n) {
      if (n < 2) return 1;
      return n * factorial(n-1);
}
      multiply PROC // input from two top values on the stack
            pop eax // pop a value from the stack to eax
            mov aux, eax // move this value to the auxiliary variable
            pop eax // pop one more value from the stack
            MUL eax, aux // multiply these two values
            ret // return the result in eax
      multiply ENDP
```

```
      factorial PROC //input n in eax
            push eax // push current value onto the stack
            dec eax // decrease the value of n
            JZ finish // if it is zero go to finish
            call factorial // otherwise call factorial
            push eax // push the result of last factorial's call to the stack
            call multiply // call multiply subroutine
            ret // return
finish: pop eax // pop the parameter from the stack into eax
            ret // return with the result in eax
      factorial ENDP // side effect?
```

L16-符号和大小表示，浮点互换

一、BCD(Binary-coded decimal) 原始十进制的逐位二进制表示,每位数四位二进制

二、sign-and-magnitude representation 符号和大小表示 最左侧是符号位，1 代表负数。此时 8bit 范围[-127,127]

三、补码 可认为 0-1000 被分为 0-499 和 500-1000，后者被分配为负数；二进制中 1 开头为负数；负数生成补码可以对正数反转后+1；负整数的**高进制补码**可以通过对其绝对值取反加一得到。

四、浮点数，为最大精度，不带前导 0，归入指数 **小数转浮点**以-123.625 为例

1. **确定符号位（Sign bit）：**因为小数是负的，符号位为 1。
2. **将整数和小数部分转为二进制：**整数−123 的二进制是 11110111111011。小数 0.625 的二进制是 0.1010.101。
3. **合并整数和小数的二进制表示：**将整数部分和小数部分合并，得到 1111011.101。
4. **将合并后的二进制数标准化：**移动小数点，使只有一个非零位在小数点的左边，得到 1.111011101×2^6
5. **确定指数部分（Exponent）：**因为小数点向右移动了 6 位，所以指数部分为 6+127=133。将 133 转为 8 位二进制得到 10000101。
6. **确定尾数部分（Mantissa）：**将标准化后的二进制数中小数点右边的部分作为尾数，得到 11101110100000000000000。**最终** −123.625 的单精浮点数：1 10000101 11101110100000000000000

浮点转小数 1、符号位 1 代表负数；2、指数部分 10000101=133-127=6；3、尾数部分加 1=1.111011101；4、计算 1.111011101×2^6=1111011.101=-123.625（勿忘符号）

L17-测试溢出

一、测试溢出：若加法的两个输入符号相同，而输出符号不同，则发生溢出。

L19-数据储存，RAM，内存

〇、有两种数据储存：Main memory 主存，Mass storage 大容量储存器

一、CPU 只能执行加载到主内存中的指令，决定了同时运行的程序数，分配给程序的空间

二、RAM: Random Access Memory.是以区别磁带 serial tape storage（线性），所有数据访问时间相同 **volatile 易失**

可看作一组被编号的存储元素(Word)，对 32 位机，每 Word 中有 32bits；1Btye(B)=8bits,1KB=1024B-MB-GB-TB

DRAM(Dynamic 动态)慢,便宜,via 电容器 capacitors,需刷新;**SRAM**(Static 静态)快,贵,via 触发电路 flip-flops,无需刷新

DRAM 需要刷新-电容器漏电-储存的数据消失-为了保留数据，必须定期刷新

三、典型值 RAM-512MB/1GB 软盘 floppy disk-1.44MB,CD-650MB,DVD-4.7GB,HDD-Hard drive~300GB（数据过时）

四、ROM 控制用只读存储 Read-Only Memory(固件 firmware)help boot up system, BIOS(basic input output system)

五、Cache(缓存)一级 8-64kb 二级 128-512kb（数据过时）同步读写 CPU<-[word]->Cache<-[block]->Main memory

六、大容量储存器 Optical disks(CD-ROM/RW,DVD…)Hard Disk Drives 寻址 10ms 10^-2s 相比 RAM10ns 10^-8s

七、采样率 CD 一个小时的大小：44.1khz*2 声道*3600s*16bit/8/1024/1024=约 605.62MB

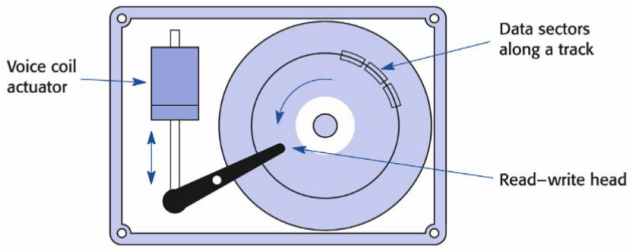
L20-内存映射，缓存

〇、主存中的每个 memory 都有自己的地址，memory 的数量取决于地址(寻址)长度**(空间长度*机器位数/8)**

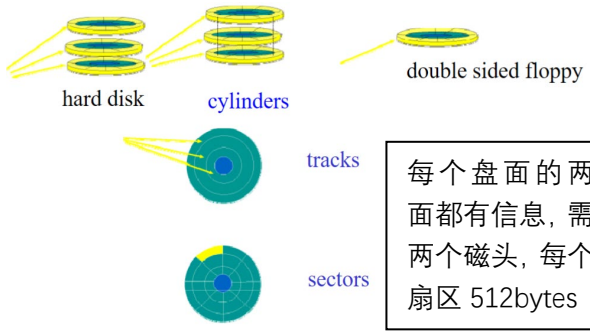
16bit-64kb, 20bit-1mb, 24bit-16mb, 32bit(奔腾)-4Gb, 64bit->17bilionGb

打印数组：

```
int main ()
{
    int sum = 0;
    int intArray [5] = {6,1,8,4,5};
    _asm {
        mov ECX, 5
        lea EDX, intArray
        myLoop:
            mov eax, [EDX]  // 使用 mov 指令加载数组元素到寄存器 eax
            add sum, eax
            add EDX, 4  // 增加数组索引
            loop myLoop
    }
    printf ("sum= %d", sum);
    return 0;
}
```



L21-储存



每个盘面的两面都有信息，需两个磁头，每个扇区 512bytes

一、**CHS**-Cylinder 柱面, Head 磁头,Sector System 扇区系统

二、**LBA**-Large Block Addressing 大数据块寻址,绝对扇区数寻址

三、**disk cache 磁盘缓存**,将主存作为 buffer,使写入操作聚集(clustered) **Java**: “flush ()” in java.io.PrintWriter.

四、检索 retrieving 文件进 RAM-加载应用,打开文件-**reading;**将 RAM 中数据复制入 secondary storage-**writing**

五、**虚拟内存 Virtual memory** 用低级储存扩展内存，磁盘上为此保留的区域称为**交换区域 swap area**

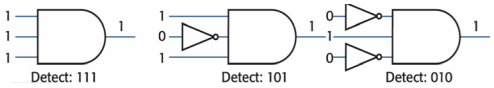
主存被分为帧 frames-一般 4kb，组成了 pages，**主存溢出的 page 与从磁盘检索到的 page 交换**

32bits 逻辑地址在 4KB 分页情况下：低 12 位用于**帧内寻址**，高 20 位作为**帧编号**

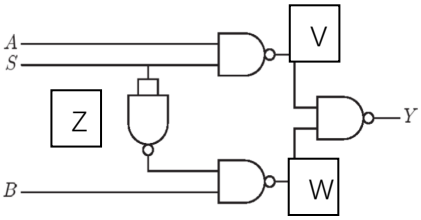
L22-数字电路，门

名称 英文	与门 AND	或门 OR	非门 NOT	与非门 NAND	或非门 NOR	异或门 XOR	同或门 XNOR
表达式	Y=A&B	Y=A|B	Y=~A	Y=~(A&B)	Y=~(A|B)	Y=A^B	Y=~A&~B | A&B
国内符号	A B & Y	A B ≥1 Y	A 1 Y	A B & Y	A B ≥1 Y	A B =1 Y	A B = Y
国际符号							
输入 A B	输出	输出	输出	输出	输出	输出	输出
0 0	0	0	1	1	1	0	1
0 1	0	1	1	1	0	1	0
1 0	0	1	0	1	0	1	0
1 1	1	1	0	0	0	0	1

三输入门 three-input gates，与门或门类似，多一个输入引脚



Selector circuit 选择器电路



S=1→Y=A；S=0→Y=B；**Y=(S&A)∨(¬S&B)**

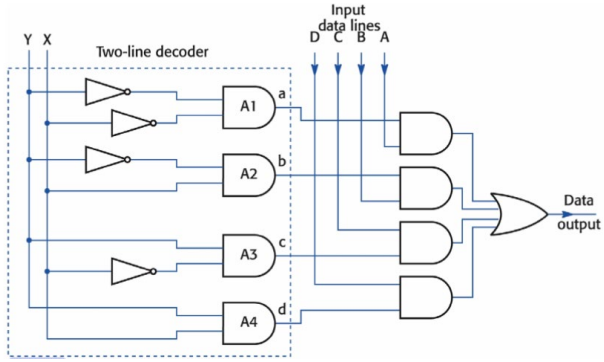
若需电路分析，从输出端向输入端分析，最后合并即可

– Y = not (V and W),where,– V = not (A and S),– W= not (Z and B),– Z = not (S and S)= not S.

得到：Y = not (not (A and S) and not (not (S) and B)).

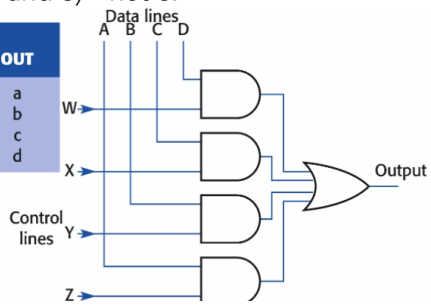
Data selector, or multiplexer 数据选择器，多路复用器→

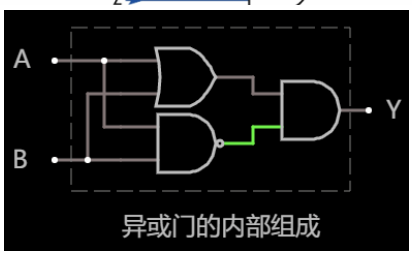
Two-line decoder 用 xy 组合选择数据↓



Selector Y X	Line d c b a
0 0	0 0 0 1
0 1	0 0 1 0
1 0	0 1 0 0
1 1	1 0 0 0

Control W X Y Z	Data A B C D	OUT
0 0 0 1	a b c d	a
0 0 1 0	a b c d	b
0 1 0 0	a b c d	c
1 0 0 0	a b c d	d

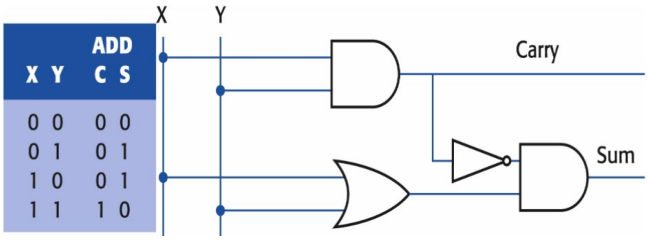




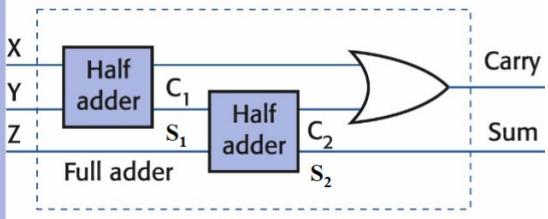
O = (A and (not X and not Y))or(B and (X and not Y))or(C and (not X and Y))or(D and (X and Y)).

L24-加法器，触发器

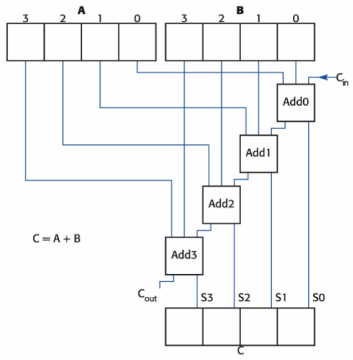
一、half adder 半加器↓



X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



二、Full adder 全加器，对于多位二进制数的加法，需处理上一阶段的进位，意味着加法器应该有三个输入↑。



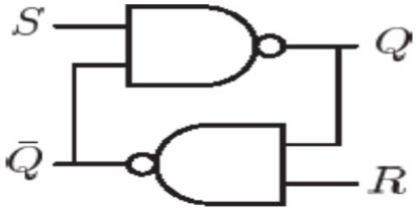
四位全加器

三、Sequential logic circuits 时序逻辑：组合 Combinational 逻辑电路-输出仅取决于同一时刻的输入，没有记忆

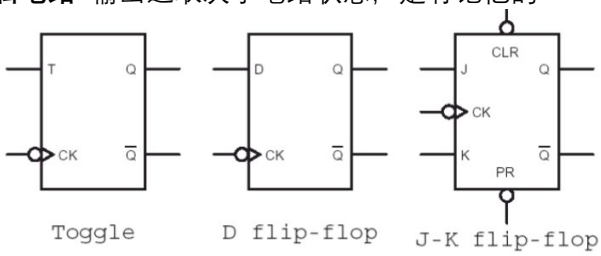
顺序 Sequential 逻辑电路-输出还取决于电路状态，是有记忆的

1、Flip-flops/latches 触发器

SR 触发器 set-reset flip-flop.



R	S	Q	状态
0	0	保持状态	无变化
0	1	1	Set
1	0	0	Reset
1	1	不稳定	禁止

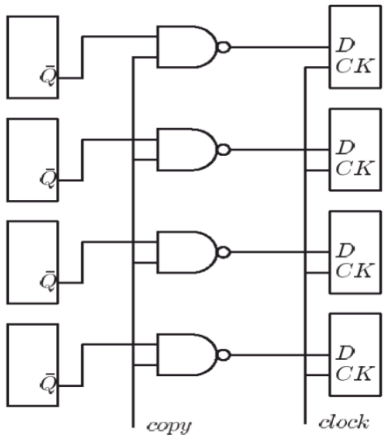


T	Q
0	Q_{prev}
1	$\bar{Q}_{prev}$

D	Q
0	0
1	1

J K	Q
0 0	Q_{prev}
0 1	0
1 0	1
1 1	$\bar{Q}_{prev}$

D 触发器 D flip-flops-可以从寄存器复制数据的电路



CK 出现时 copy 才有作用 copy 为 1 时、出现 ck 时会把左边复制给右边。

L3 附录-几种错误的可能形式

Compile-time Error (编译时错误)：在代码编译阶段就被检测到，阻止了程序的编译过程。典型的编译时错误包括语法错误、类型错误、未声明的变量等。例如在 C++中，若有一个变量未声明而直接使用，编译器将报编译时错误。

Link-time Error (链接时错误)：在代码编译成功后，但在链接时出现问题。包括找不到函数或变量的定义、重复的符号等。例如在 C 语言中，若有一个函数被声明但未定义，链接器将报告链接时错误。

Load-time Error (加载时错误)：发生在程序被加载到内存时。包括依赖的库文件不存在、动态链接库（DLL）版本不匹配等。例如在使用动态链接库的情况下，如果所需的 DLL 文件不存在，将导致加载时错误。

L13 附录-汇编子程序：

7.1.1 子程序 subroutine 子程序是代码的一部分，可以在正在执行的程序中重复使用。

7.1.2 优点：Save the effort in programming. 节省编程的工作量 Reduce the size of programs. 减少程序的大小 Share the code.共享代码 Encapsulate, or package, a particular activity 封装或包装特定的活动-向用户隐藏复杂性 Provide easy access to tried and tested code. 能够方便测试代码

7.1.3 汇编子程序：PROC==过程开始 ENDP==过程中止 指令 RET 更改控制，导致从 CALL 指令后面的点继续执行。

7.1.4CALL 指令的操作：将 EIP(指令指针)的当前值记录为返回地址。将所需的子例程放入 EIP(指令指针)，因此要执行的下一条指令是子例程的第一条指令。

RET 执行操作：弹出堆栈中储存的最后一个地址并将其放入 EIP。

7.2.1 参数的两种形式：值参数 value parameter 数字值或者 ASCII 码

引用参数 reference parameter 将内存地址作为参数传递给子程序

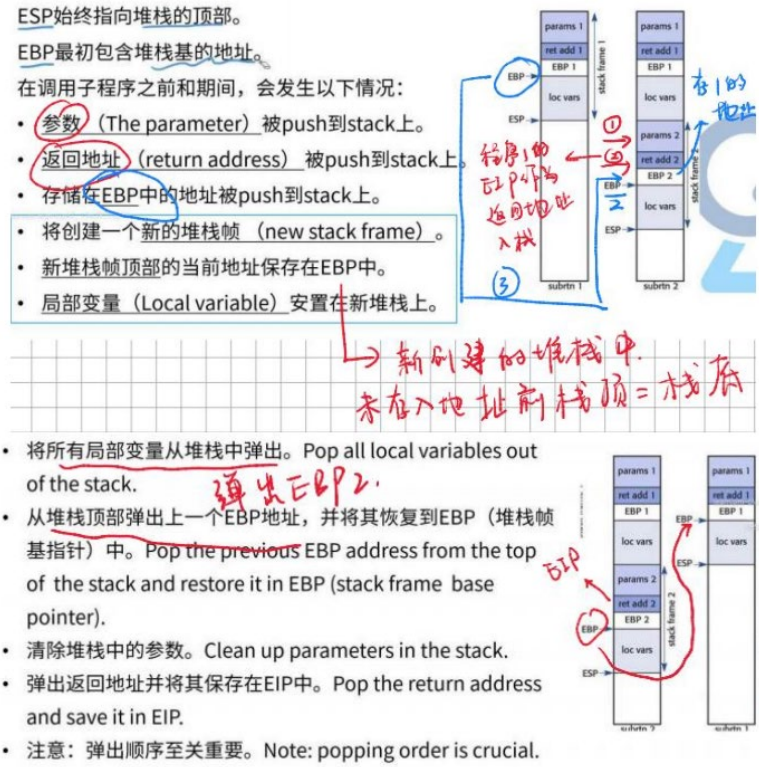
7.2.2 传递参数 寄存器 最简单的方法 约束性：个数有限，不能无限制的传递参数，有的寄存器有着特定的作用。堆栈 大多数计算机的解决方案是使用堆栈传递参数并保留局部变量。

7.2.3 堆栈帧 stack frame 堆栈中保存与一个子例程调用相关的所有数据的区域称为堆栈帧，包括：子程序的参数返回地址 局部变量-执行子程序期间储存在 stack 中

7.2.4 两个寄存器用于处理堆栈帧

EBP:堆栈帧基指针 指示堆栈帧底部 ESP 堆栈指针 保存顶部地址

7.2.5 子程序完成工作后



8.1.1 递归子例程或过程在某些情况下可能会调用自身来执行某些辅助任务。

8.1.2 递归实现阶乘 过程中需要一个辅助程序

multiply 获取堆栈上的两个顶部值，将它们相乘并在 eax 寄存器中返回结果。

缺点：堆栈需要改变两次 EAX 原始内容被替换

L13 附录-printf 调用

C 内联汇编在调用函数 call printf("%d %d \n\n", i, j);前，向栈中压的参数个数 = 被逗号分隔开的数量 = 逗号数+1;

L21 附录-CPU 访问延迟

若 CPU 需访问 5 页内存且访问概率相等，访问主存耗时 5ns，从磁盘读/写一页耗时 5000ns，假设因空间不足，一页读完后必须交换，不得使用 cache；若内存中已有一页，访问 5 页的平均时长为：1/5*5+4/5*(5+5000*2) =8005ns

----------未经作者同意，请勿商用----------

https://blog.csdn.net/xxsjyjdw https://blog.csdn.net/xxsjyjdw/article/details/135515051